

ORGANIZATION AND GRAPHICAL REPRESENTATION OF A VARIABLE

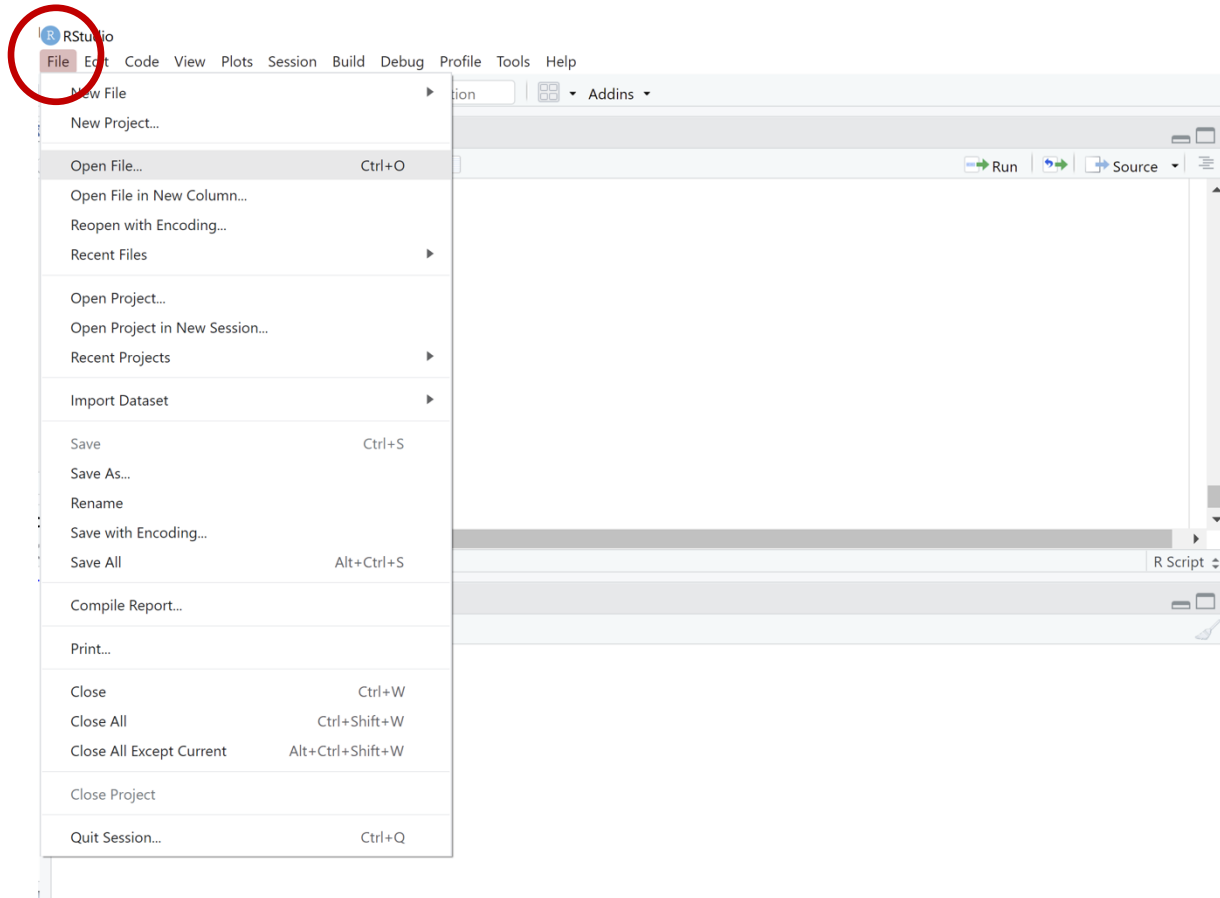
Analysis of data collected on a single variable

To describe the tools essential for organizing data collected on a single variable, we refer to the information in the **Loyalty.Rdata** dataframe, containing data on some characteristics of a sample of customers and on their interactions with a multi-store over a specific period:

- ▶ **Customer:** client's identifier
- ▶ **Age:** client's completed age in years (at the time of the survey)
- ▶ **Income:** client's income class
- ▶ **Payment:** most frequently used payment method during the considered period
- ▶ **Dept:** department where the client purchased most during the considered period
- ▶ **Nr_visits:** number of visits during the considered period
- ▶ **Amount:** total amount spent during the considered period
- ▶ **Profitability:** customer's profitability as assessed by the company
- ▶ **Recommend:** (stated) propensity to recommend the multi-store
- ▶ **Amount_V:** average amount spent per visit
- ▶ **Recommend.F:** factor derived and created based on variable Recommend

Dataframe: loading a file created with R

To load a file that was generated with R and has the ".Rdata" extension, open the **File** menu, choose the **Open File** option and select the desired ".Rdata" file from your file system.



The chosen ".Rdata" file will be promptly loaded into the Environment section within Rstudio.

Variables, data, and categories

Data for a variable can be collected on the entire set of N cases constituting a **population** or on a subset of n cases selected as a **sample** drawn from a population.

Let us consider **data** collected on a sample of n units:

$$x_1, x_2, \dots, x_n$$

Raw data are not easy to analyse and interpret, and it is necessary to **organize them** effectively to emphasize their salient information value.

This can be accomplished using **tables and charts**, which are built taking into account:

- ▶ The **type of data, qualitative (nominal or ordinal) or quantitative (discrete or continuous)**
- ▶ The **number K of distinct values** (or categories, or levels) observed in data:

$$x^*_1, x^*_2, \dots, x^*_K$$

Organization and graphical representation of a variable

Qualitative variables

Tables and graphs for qualitative variables

Qualitative variables typically take a **limited number of distinct categories**.

The data are organized into a **frequency distribution**, a table showing:

- ▶ The observed distinct **categories** taken by the variable
- ▶ The **absolute** frequencies (count of cases in each category)
- ▶ The **relative** frequencies (proportion of cases in each category, obtained as the ratio between the absolute frequency and the total number of cases) and/or their corresponding **percentages**

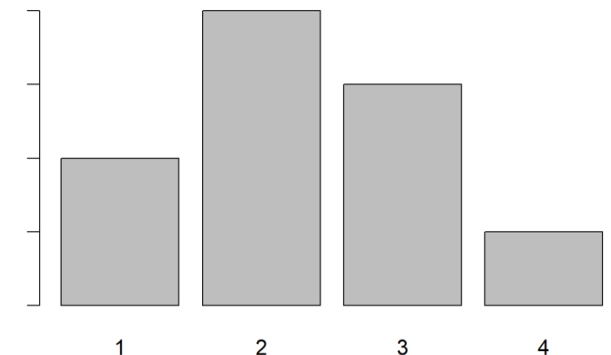
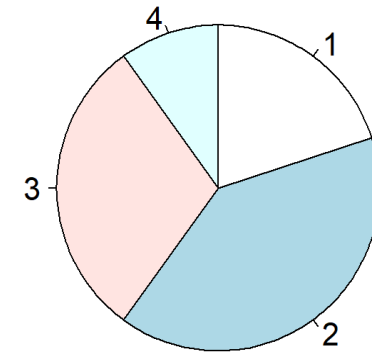
Variable X	Absolute Fr. f_k	Relative Fr. $p_k = f_k/n$	Percentage
x^*_1	f_1	p_1	$100p_1$
x^*_2	f_2	p_2	$100p_2$
x^*_3	f_3	p_3	$100p_3$
...	...	...	...
$x^*_{\textcolor{red}{K}}$	$f_{\textcolor{red}{K}}$	$p_{\textcolor{red}{K}}$	$100p_{\textcolor{red}{K}}$
	Total=n	Total=1	Total=100

Example related to a **sample** of size (number of cases) n .
When referring to a population, the entire count of cases corresponds to the population size, which is indicated by N .

Tables and graphs for qualitative variables

To graphically represent the frequency distribution of a qualitative variable, the following graphs are commonly used:

- ▶ **Pie chart:** a circle divided into segments whose areas correspond to the frequencies of the observed categories. The graph provides no information about the possible order of the categories. In some cases, it might be challenging to discern the possible differences between the frequencies of distinct categories.
- ▶ **Bar chart:** a set of bars with equal width and heights equal to the frequencies of the observed categories. This graph allows to visualize the order of the categories, and is particularly suitable for ordinal variables. It can also be used for nominal variables, provided it doesn't lead to the wrong conclusion that the categories have an order .



Rstudio: table() [OPTIONAL]

The frequency distribution of a variable can be easily obtained using the function `table(x)`, with a unique argument, `x`, which is the (name of the) vector containing the data or the name of the column in a dataframe, as in the example below

```
> table(Loyalty$Dept)
Accessories      Clothing  Home/Textiles      Kids
           27          39          16          19
Perfumery Sports&Outdoors
           17           7
```

The function prints the distribution of **absolute frequencies**.

Relative frequencies and percentages can be obtained easily; if, as in this case, **there are no missing values**:

```
> table(Loyalty$Dept) / nrow(Loyalty)
Accessories      Clothing  Home/Textiles      Kids
      0.216          0.312          0.128          0.152
Perfumery Sports&Outdoors
      0.136          0.056
```

Rstudio: `distr.table.x()`

To build a table reporting different types of frequencies, the `distr.table.x()` function can be used. Focusing only on the arguments relevant in this simple case, the syntax of the function is as follows:

```
distr.table.x(x, freq= c("counts", "proportions"), total=TRUE, data)
```

where:

- ▶ **x** is a vector or factor (e.g., `x=Loyalty$Dept`) or the name of a column in the dataframe specified in **data** (e.g., `x=Dept` with `data=Loyalty`)
- ▶ **freq** is a vector listing the frequencies to be displayed in the table (more options are possible). Available options are `counts`, `percentages`, `proportions` (possibly abbreviated, e.g. `prop`); if no input is provided, `counts` and `proportions` are reported (other options will be discussed later)
- ▶ **total**: **logical** value specifying whether the totals (sum of specified frequencies) should also be reported in the table (`total=TRUE`, default) or not (`total=FALSE`).

Rstudio: `distr.plot.x()`

Different functions are available in R to graphically display a table. To simplify the syntax, it is possible to use the `distr.plot.x()` function in package `UBStats`. In the case considered so far, the syntax of the function is:

```
distr.plot.x(x, freq="counts", plot.type, bw=FALSE, data)
```

Where:

- ▶ **x** a vector or factor (e.g., **x=Loyalty\$Dept**) or the name of one column in the dataframe specified in **data** (e.g., **x=Dept** with **data=Loyalty**)
- ▶ **freq** indicates which type of frequency (only one option is allowed) should be reported in the plot. Available options are **counts**, **percentages**, **proportions** (possibly abbreviated, e.g. **prop**); if no input is provided, the plot will display counts (other options will be discussed later)
- ▶ **plot.type** is the type of plot to be generated (e.g. **pie**, **bars**)
- ▶ **bw** is a logical value indicating whether the plot should be coloured in shades of grey (**bw=TRUE**) or not (**bw=FALSE**, default). In the latter case a standard colours palette is used, although it is possible to specify the desired colours using the additional argument **color** (details in the Manual on R)

Hands on: tables & plots for qualitative variables

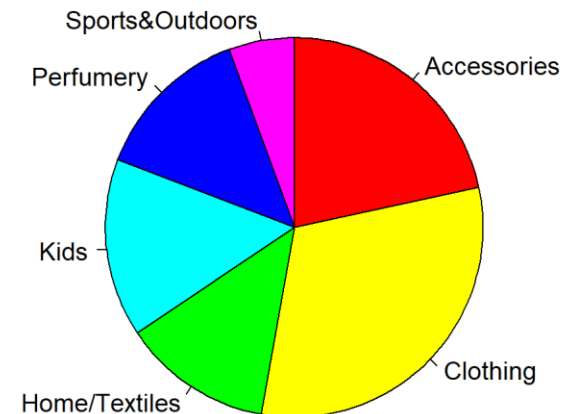
Obtain the table displaying both the relative and the absolute frequencies for the variable **Dept** in the dataframe **Loyalty** and provide its graphical representation

```
distr.table.x(x=Dept, freq= c("counts","prop"), total=T, data=Loyalty)
distr.table.x(x=Dept,data=Loyalty) # same: freq and total -> defaults
distr.plot.x(x=Dept,freq="proportions",plot.type="pie", data=Loyalty)
```

```
> distr.table.x(x=Dept, data=Loyalty)
```

Dept	Count	Prop
Accessories	27	0.22
Clothing	39	0.31
Home/Textiles	16	0.13
Kids	19	0.15
Perfumery	17	0.14
Sports&Outdoors	7	0.06
TOTAL	125	1.00

Pie chart: Dept



Pie chart: The chart provides information only on the relative importance of each category.

Hands on: tables & plots for qualitative variables

Obtain the table displaying both the relative and the absolute frequencies for the variable `Recommend` in the dataframe `Loyalty` and provide its graphical representation

```
distr.table.x(x=Recommend,data=Loyalty) # freq and total -> defaults
```

```
> distr.table.x(x=Recommend, data=Loyalty)
```

Recommend	Count	Prop
Likely	34	0.27
Neutral	31	0.25
Unlikely	30	0.24
Very Likely	17	0.14
Very Unlikely	13	0.10
TOTAL	125	1.00

The order of the categories in the table is standard (alphabetical), even if the variable is ordinal!!!

For ordinal variables, it is strongly advised (always) to use a pre-defined factor to appropriately order their levels.

Instead of `Recommend` it is sensible to analyze the factor `Recommend.F` already included in the dataframe, which was created as follows

```
> Loyalty$Recommend.F<-factor(Loyalty$Recommend,  
+                             levels=c("Very Unlikely","Unlikely","Neutral",  
+                                     "Likely","Very Likely"))
```

Hands on: tables & plots for qualitative variables

Obtain the table displaying both the relative and the absolute frequencies for the variable **Recommend** in the dataframe **Loyalty** and provide its graphical representation

```
distr.table.x(x=Recommend.F ,data=Loyalty) # freq and total -> defaults  
distr.plot.x(x=Recommend.F,freq="perc",plot.type="bars", data=Loyalty)
```

```
> distr.table.x(x=Recommend.F, data=Loyalty)
```

Recommend.F	Count	Prop
Very Unlikely	13	0.10
Unlikely	30	0.24
Neutral	31	0.25
Likely	34	0.27
Very Likely	17	0.14
TOTAL	125	1.00



Bar chart: the horizontal axis reports categories arranged in increasing order. Note that this **axis has no numerical meaning**: these categories are not numbers, and quantifying the distances among them is not possible or meaningful. To avoid misinterpretation, bars are displayed at the same distance one from another.

**Organization and representation and graphics
of a variable**

Quantitative variables

Tables and graphs for quantitative variables

For quantitative (numerical) variables, it is important to consider that:

- ▶ The number of distinct values taken by **discrete variables** can be small (e.g., count of store visits in a certain period), as well as relatively large (e.g., customer ages in year)
- ▶ **Continuous variables** typically take a different value for each observation, with a consequent number of observed distinct values either equal to or very close to the total number of cases

Nr of visits	f_k	p_k
1	18	0.144
2	48	0.384
3	35	0.280
4	16	0.128
6	6	0.048
7	2	0.016
Total	125	1

(6 distinct values)

Age	f_k	$100p_k$
25	2	1.60%
27	2	2.40%
28	2	1.60%
29	2	1.60%
30	2	1.60%
...	...	...
75	1	0.80%
76	1	0.80%
80	1	0.80%
Total	n=124	100%

(50 distinct values)

Amount	f_k	p_k
25.5	1	0.008
25.6	1	0.008
25.9	1	0.008
26.6	1	0.008
27.8	1	0.008
29	1	0.008
30	1	0.008
...	...	...
75.1	1	0.008
76.9	1	0.008
77.3	1	0.008
79.8	1	0.008
85.7	1	0.008
86.6	1	0.008
Total	n=125	100%

(118 distinct values)

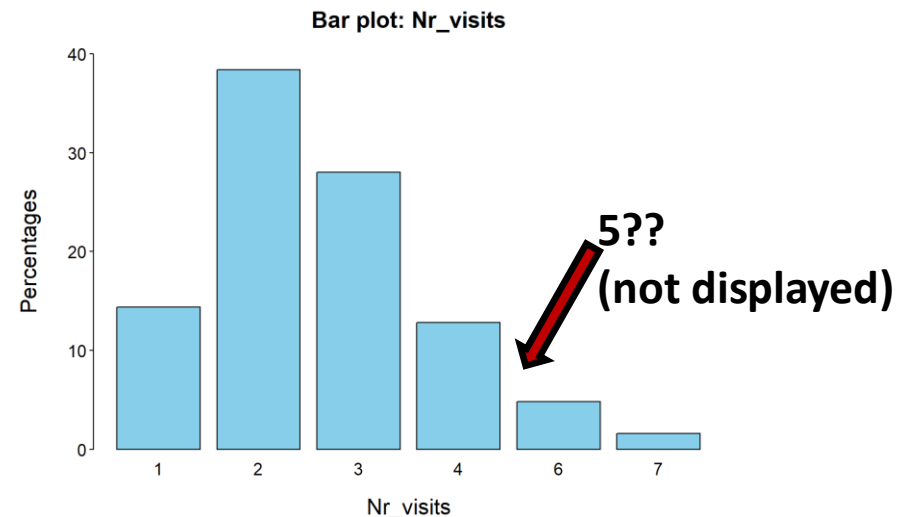
It is evident that when the number of distinct values is high, the descriptive effectiveness of the frequency distribution is minimal, and it is therefore necessary to simplify the table.

Hands on: tables & plots-discrete variables (few distinct values)

Obtain the table displaying the relative and the absolute frequencies and the percentages for the variable `Nr_visits` in the dataframe `Loyalty` and provide its graphical representation

```
distr.table.x(x=Nr_visits, freq= c("counts", "prop", "perc"), data=Loyalty)
distr.plot.x(x=Nr_visits, freq="perc", plot.type="bars", data=Loyalty)
```

Nr_visits	Count	Prop	Percent
1	18	0.14	14
2	48	0.38	38
3	35	0.28	28
4	16	0.13	13
6	6	0.05	5
7	2	0.02	2
TOTAL	125	1.00	100



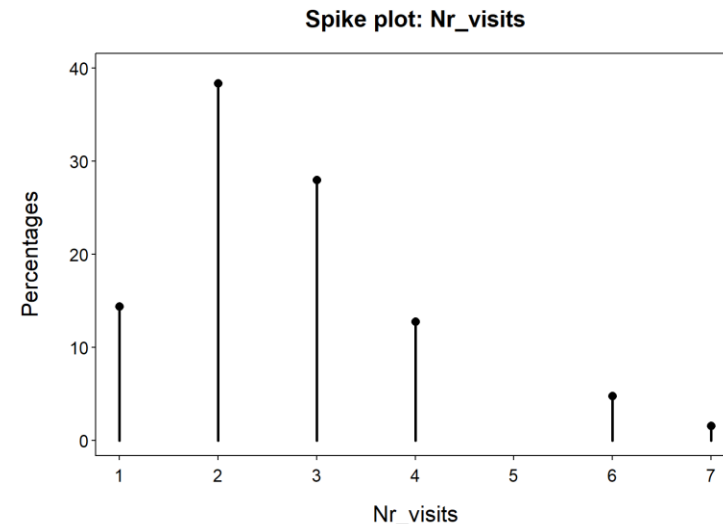
Bar chart: the variable is numerical; therefore in this case the values on the horizontal axis are numbers! Caution is needed when some of the values between the minimum and the maximum values are not observed, because the plot can lead to misinterpretation, giving a false impression of equally spaced values.

Hands on: tables & plots-discrete variables (few distinct values)

Obtain the table displaying the relative and the absolute frequencies and the percentages for the variable `Nr_visits` in the dataframe `Loyalty` and provide its graphical representation

```
distr.table.x(x=Nr_visits, freq= c("counts", "prop", "perc"), data=Loyalty)
distr.plot.x(x=Nr_visits, freq="perc", plot.type="spike", data=Loyalty)
```

Nr_visits	Count	Prop	Percent
1	18	0.14	14
2	48	0.38	38
3	35	0.28	28
4	16	0.13	13
6	6	0.05	5
7	2	0.02	2
TOTAL	125	1.00	100



The **Spike (or stick) plot**: takes into account both the observed values and the **distances** among them. Each **observed** value, on the horizontal axis, is associated a vertical stick whose height corresponds to its frequency.

Frequency table: numerical variables (many distinct values)

For continuous variables, and also for discrete variables taking a large number of **distinct values**, the frequency table becomes hard to interpret and **ineffective**; indeed the table has many rows and many values are characterized by possibly low frequencies.

In these cases **it is convenient to** classify data into **interval classes**.

- ▶ Intervals should be **contiguous** and **non-overlapping**, and should cover the entire set of **observed values** (thus, **exhaustive and mutually exclusive intervals** should be used)
- ▶ The endpoints of the intervals should be clearly defined. By convention, the intervals' lower endpoint is included in the interval, while the upper endpoint is excluded (except for the **final interval**)

In this case, the **frequency distribution** is a table listing the interval classes and associating to each class its **absolute** frequency (number of cases with values within the interval), and/or its **relative** frequency (proportion of cases with values within the interval), and/or the corresponding **percentage**.

Interval classes with the same size

To classify data into **intervals** (classes) having the **same width** w , w can be obtained as:

$$w = \frac{\text{Max} - \text{Min}}{\text{Number of classes}}$$

where **Min** and **Max** indicate the smallest and largest observed values, respectively.

The resulting width can be rounded to obtain a simpler classification (with classes whose extremes are integers, or are conveniently rounded for example).

Interval classes with the same size

Example: Consider data on the time needed to close 20 service requests tickets (issues related to a specific operation):

24, 35, 17, 21, 24, 37, 26, 46, 58, 30

32, 13, 12, 38, 41, 43, 44, 27, 53, 27

You want to group data into **5** intervals of **equal width**.

- ▶ Order the data:

12, 13, 17, 21, 24, 24, 26, 27, 27, 30, 32, 35, 37, 38, 41, 43, 44, 46, 53, 58

- ▶ Determine the size of the 5 intervals:

$$w = \frac{\text{Max} - \text{Min}}{\text{Number of classes}} = \frac{(58 - 12)}{5} = 9.2$$

- ▶ To simplify the classification, create intervals of width 10 starting from 10 (instead of 12).

Interval classes with the same size

R: `dist.table.x(tickets, breaks = c(10, 20, 30, 40, 50, 60))`

Example: Time needed to close 20 service requests tickets.

12, 13, 17, 21, 24, 24, 26, 27, 27, 30, 32, 35, 37, 38, 41, 43, 44, 46, 53, 58

Data grouped into 5 classes with the same width ($w = 10$)

Ticket (Time)	Absolute Fr. f_k	Relative Fr. p_k	Percentage $100p_k$
[10, 20)	3	0.15	15
[20, 30)	6	0.30	30
[30, 40)	5	0.25	25
[40, 50)	4	0.20	20
[50, 60]	2	0.10	10
Total	20	1.00	100

frequency densities
 $c_k = \frac{p_k}{w_k}$
 $c_1 = 0.015$
 $c_2 = 0.03$
 $c_3 = 0.025$
 $c_4 = 0.02$
 $c_5 = 0.01$

concentration of relative frequency

EXAMPLE:

class

[0, 5)

[5, 15)

[15, 25)

[25, 55]

f

20

30

20

30

p

0.20

0.30

0.20

0.30

w

5

10

10

30

c

0.04

0.03

0.02

0.01

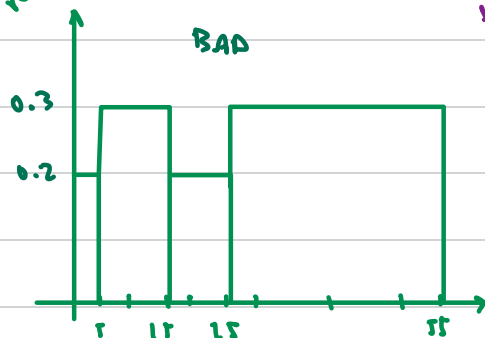
always

construct histogram

on freq.

densities

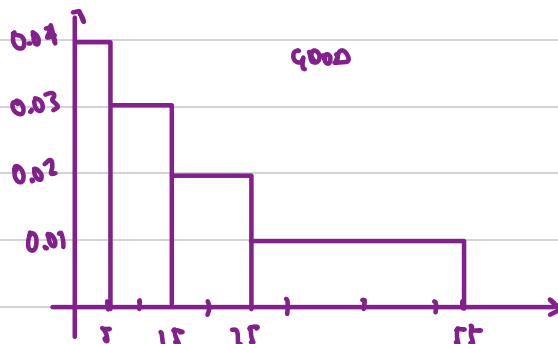
rel. freq.



MISLEADING REPRESENTATION!

It does not show where data is concentrated

freq. dens.



miraculous histogram! It will have a pretty consistent shape regardless the classes I set

TAKE HOME MESSAGE: histograms have FREQUENCY DENSITIES on the y-axis for robust and relevant representation of data

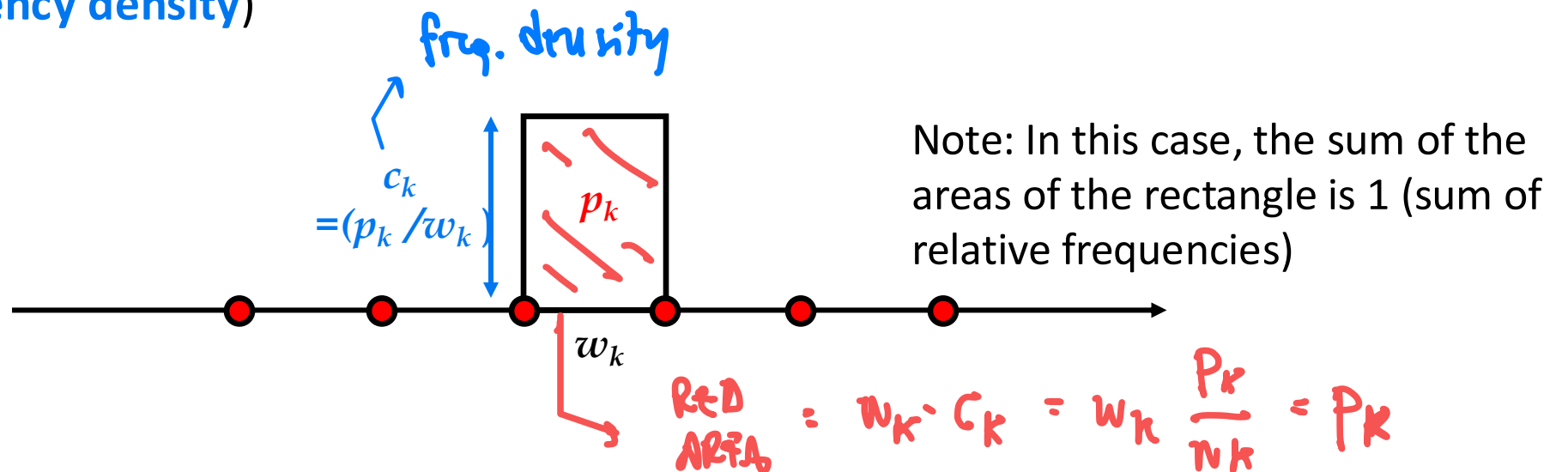
① The shape of the hist is consistent if we change the classes

② Area of the histogram correspond to relative frequencies

Histogram: classes of equal size

The frequency distribution for data organized into intervals can be graphically represented by a **Histogram**.

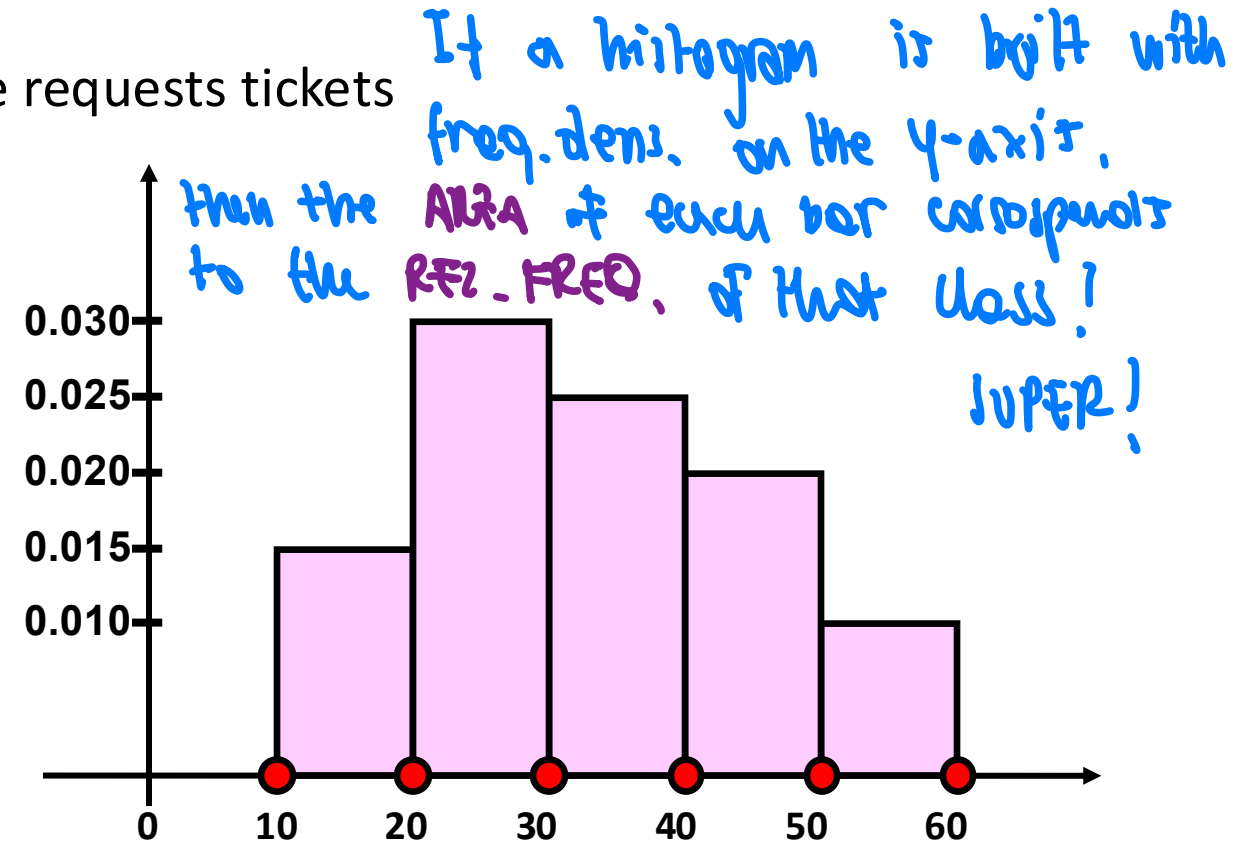
- ▶ The endpoints of the intervals are reported on the horizontal axis
- ▶ On each interval a rectangle is created having:
 - ▶ **base** = interval **width** (for the k -th interval: w_k)
 - ▶ **area** = **frequency (usually, relative frequency)** of the interval (for k -th interval: p_k)
 - ▶ → **height** = area/base=area/width (for the k -th interval: $c_k = p_k/w_k$, also called **frequency density**)



Histogram: classes of equal size

Example: Time needed to close 20 service requests tickets

Ticket (Time)	Absolute Fr. f_k	Relative Fr. p_k	Density $c_k = p_k / 10$
[10, 20)	3	0.15	0.015
[20, 30)	6	0.30	0.030
[30, 40)	5	0.25	0.025
[40, 50)	4	0.20	0.020
[50, 60]	2	0.10	0.010
Total	20	1.00	



On the y-axis the density of each class is reported.

The area of the rectangle built on each class corresponds to the relative frequency of the class, and the areas of all the rectangles sum up to 1.

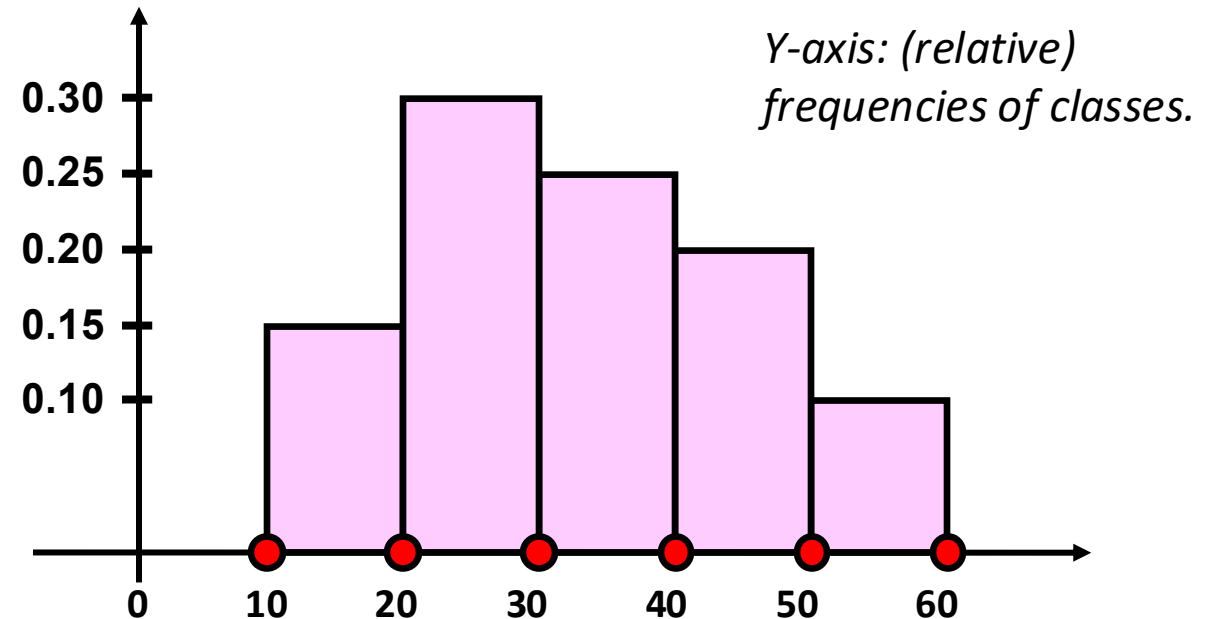
A hist. is an approximated representation of data. We are assuming the obs within each class are uniformly distributed

Histogram: classes of equal size

For intervals with the same width, w , the **densities** are calculated by dividing the relative frequencies by the **same width**, $c_k = p_k / w$. **In this case**, the histogram **may** be built using rectangles whose heights correspond to the intervals' **frequencies**. However, the area of each rectangle will be $(p_k \cdot w)$ instead of p_k , and therefore the **total area will no longer be 1**.

Ticket (Time)	Absolute Fr. f_k	Relative Fr. p_k	Density $c_k = p_k / 10$
[10, 20)	3	0.15	0.015
[20, 30)	6	0.30	0.030
[30, 40)	5	0.25	0.025
[40, 50)	4	0.20	0.020
[50, 60]	2	0.10	0.010
Total	20	1.00	

This histogram provides a correct visualization of the variable's distribution (in classes), but the sum of the areas is not 1 but 10 (common width of the classes)



Rstudio: `distr.table.x()` and `distr.plot.x()`-with breaks

To create a table and to draw a histogram for numerical data grouped into interval classes, you can use the `distr.table.x()` and `distr.plot.x()` functions by specifying intervals' endpoints using the `breaks` argument and selecting `"histogram"` as plot type.

```
distr.table.x(x, freq= c("counts", "prop", "dens"), breaks, data)
```

```
distr.plot.x(x, freq="counts", plot.type="histogram", breaks, data)
```

where, in both functions:

- ▶ Using `freq` one can request to report densities (`"densities"`) in the frequency table and to use them in histograms. For intervals with different width the histogram is always built using densities
- ▶ The `breaks` argument allows to specify the desired classification in two ways:
 - ▶ If `breaks` is a single number (e.g., `breaks=5`), it indicates the **number of intervals with equal width** to use to classify data
 - ▶ If `breaks` is a vector of **increasing** values (e.g., `breaks=c(10, 20, 30, 40, 50, 60)`), it specifies the intervals' endpoints (close on the left and open on the right, except for the last interval). To cover the entire range of values, the min and max observed values must be included between the first and last breaks. However, it is possible to specify breaks covering only partially the range of observed values to build the table or the plot limiting attention to a reduced or specific range of values

Hands on: tables & plots - numerical variables

Obtain the table of absolute and relative frequencies and densities for the time needed to close 20 tickets using 5 classes of equal width and the intervals 10-20, 20-30,...,50-60

```
# defining the vector
Ticket<-c(24,35,17,21,24,37,26,46,58,30,32,13,12,38,41,43,44,27,53,27)
distr.table.x(x=Ticket,freq=c("counts","prop","dens"),breaks=5)
distr.table.x(x=Ticket,freq=c("counts","prop","dens"),breaks=c(10,20,30,40,50,60))
```

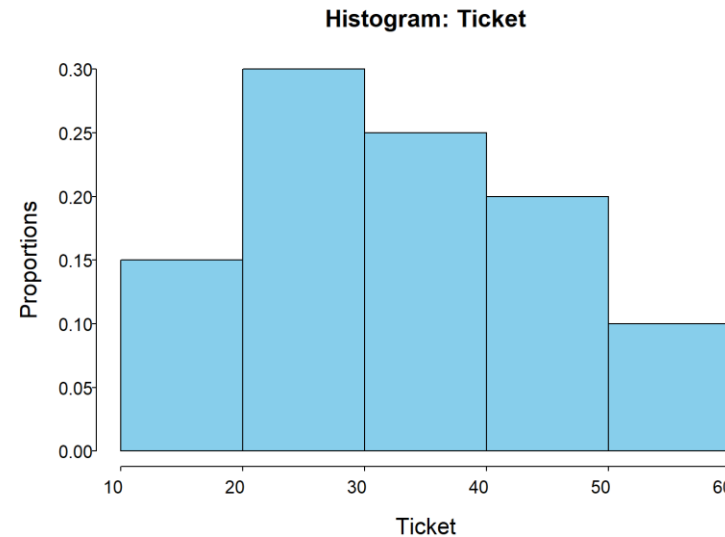
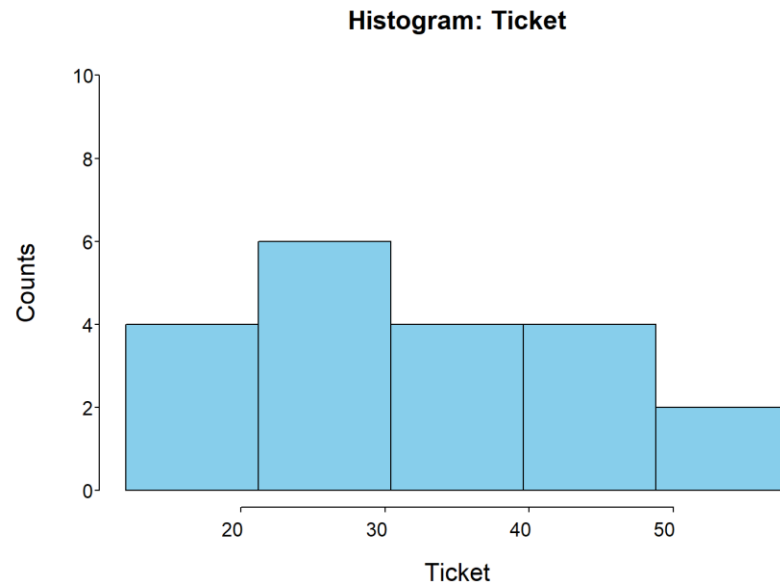
Ticket	Count	Prop	Density
[12,21.2)	4	0.2	0.02174
[21.2,30.4)	6	0.3	0.03261
[30.4,39.6)	4	0.2	0.02174
[39.6,48.8)	4	0.2	0.02174
[48.8,58]	2	0.1	0.01087
TOTAL	20	1.0	

Ticket	Count	Prop	Density
[10,20)	3	0.15	0.015
[20,30)	6	0.30	0.030
[30,40)	5	0.25	0.025
[40,50)	4	0.20	0.020
[50,60]	2	0.10	0.010
TOTAL	20	1.00	

Hands on: tables & plots - numerical variables

Provide a graphical representation of the distribution of the time needed to close 20 tickets using 5 classes of equal width and the following intervals: 10-20, 20-30,..., 50-60

```
distr.plot.x(x=Ticket,freq=c("counts"),plot.type="hist", breaks=5)  
distr.plot.x(x=Ticket,freq=c("prop"),plot.type="hist",breaks=c(10,20,30,40,50,60))
```



Histograms: Since in both cases the classes have the same width, the histogram can be built using the absolute or the relative frequencies

Interval classes with the same width: the number of classes

How many intervals?

The choice of the number of intervals (usually between 5 and 30) depends both on the number of observations and on the specific features of the variable. Statistical software usually automatically suggests a division into intervals having the same width. However, there is no optimal criterion for determining the number of classes.

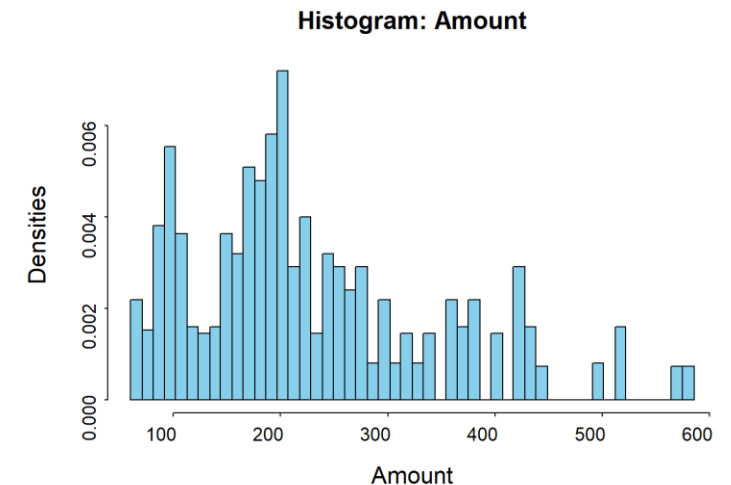
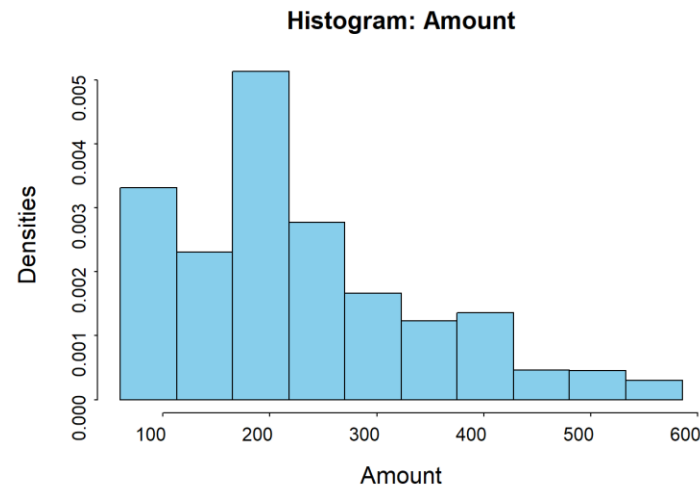
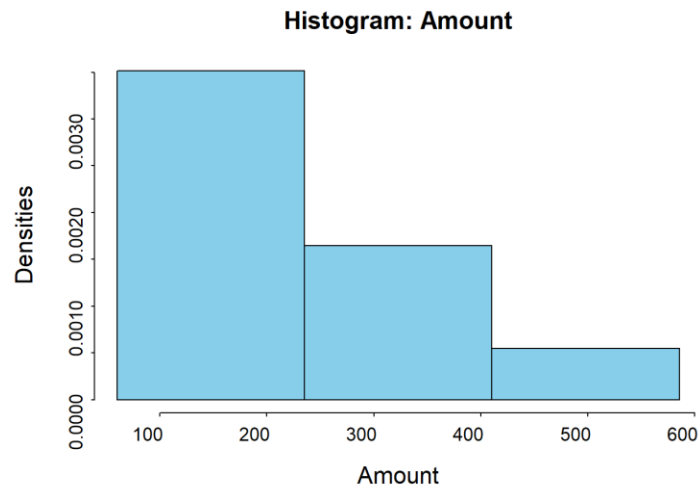
In practice, it is recommended to compare histograms created using a different number of classes keeping in mind that the histogram should:

- ▶ Highlight distinctive features and disparities within the data, avoiding an excessive number of intervals with the same (low) frequency or an insufficient number of classes that concentrate data into a few intervals with very high frequency, with a consequent difficulty in assessing the different “concentration” of data across specific intervals
- ▶ Effectively depict the distribution’s shape, symmetrical or asymmetrical, and the data’s spread (presence of any “tails”)

Hands on: tables & plots – numerical variables

Use histograms with 3, 10, and 50 interval classes of equal width to describe the distribution of the variable **Amount** within the **Loyalty** dataframe. Comment results

```
distr.plot.x(x=Amount, freq="densities", plot.type="histogram", breaks=3, bw=F,  
            data=Loyalty)  
# use the same syntax also for the 10 and 50 interval classes
```

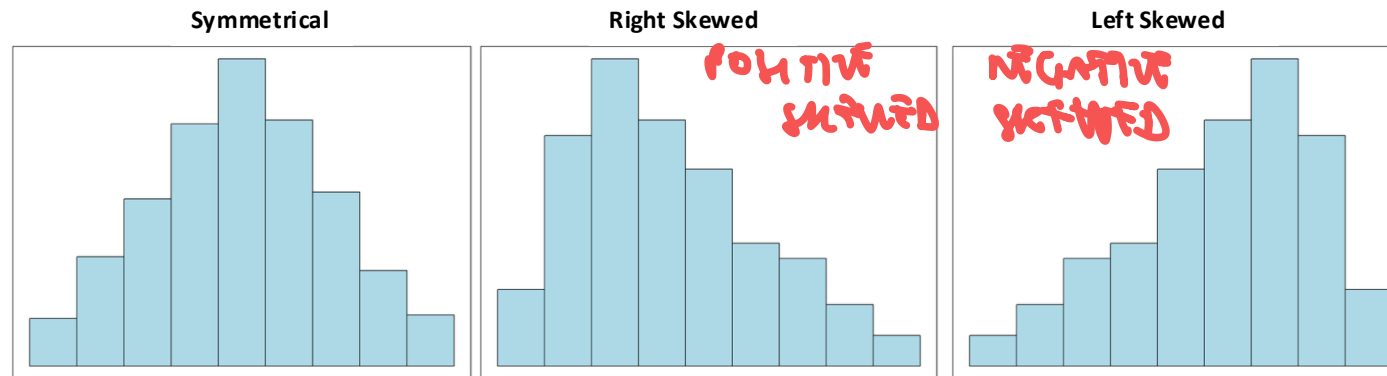


GOAL: Convey the SHAPE of the DISTRIBUTION

Histogram: shape of distribution

Relevant attributes of a distribution include its symmetry (or asymmetry) and the presence of tails

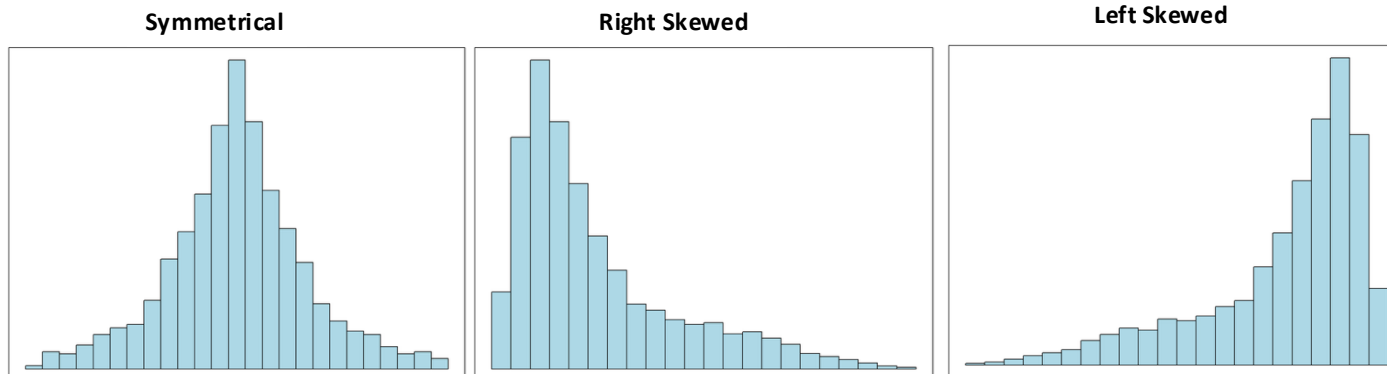
SHORT
TAILS



Symmetric distribution:
data evenly spread around the center of the histogram.

Asymmetric or skewed distributions:
Have a tail that extends primarily in one direction

LONG
TAILS



All distributions, both **symmetrical** and **skewed**, can have particularly **long tails**.

The number of classes should be chosen so that the histogram effectively highlights and captures the characteristics of the variable's distribution, including symmetry and the presence of tails

Interval classes of different widths

Frequently, most of the data tend to cluster within a relatively narrow range of values, whereas few values are spread out across a much larger range, resulting in long tails in the distribution.

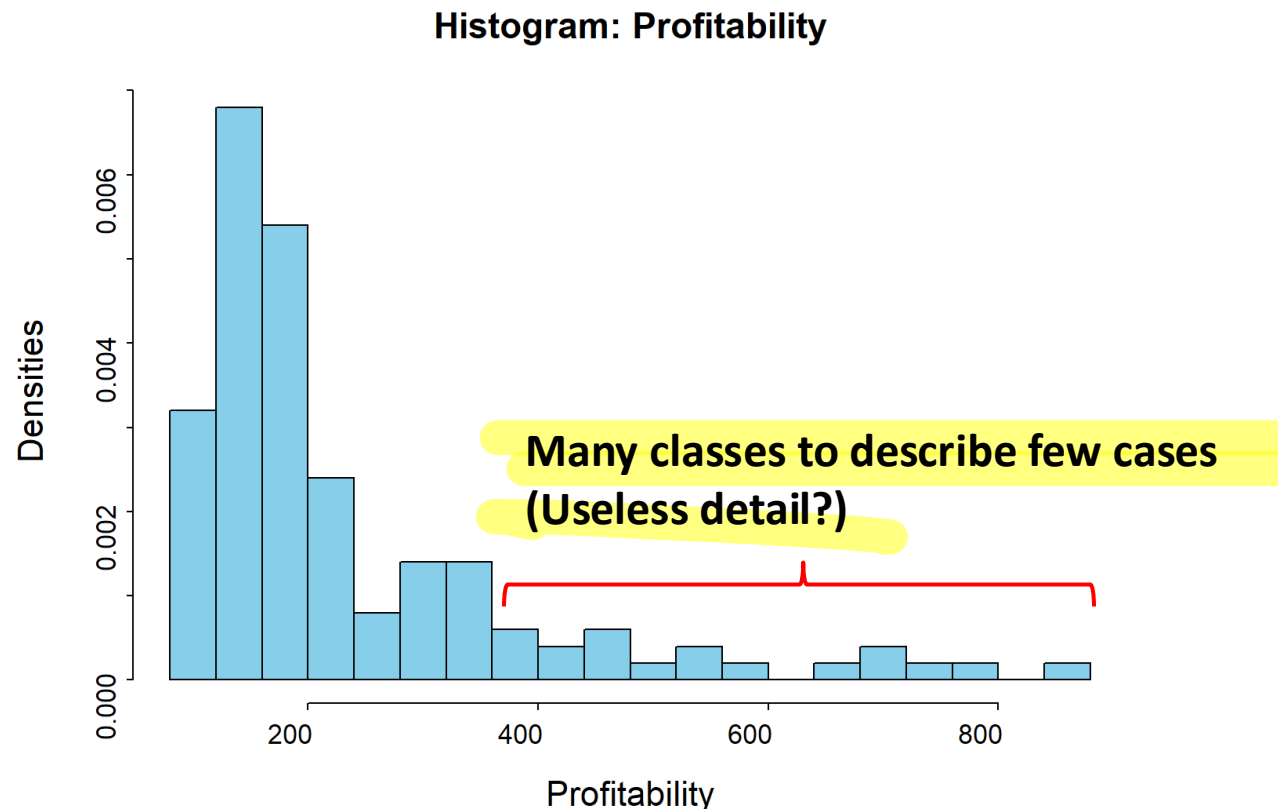
To simplify the representation of the frequency distribution in these situations, thus avoiding to use many classes (some with a possibly very low frequency) to properly represent the distribution of a numerical variable, it is sensible to use intervals with **different widths**. Such intervals can offer more detailed insight where actually needed, that is on the range where the majority of data points are concentrated, grouping all the other values in large classes.

In this case, the heights of the rectangles cannot be the relative or absolute frequencies, because the intervals have **different widths**. The histogram **must necessarily be built** using the **frequency densities**, denoted as c_k , computed as the ratios between the relative frequencies and the interval widths $c_k = p_k / w_k$.

Interval classes of different widths

Example: Variable **Profitability** in the dataframe **Loyalty** (min=81.8, max=847).
Classification into 20 classes of equal width ($w = (847-82)/20 = 38.25 \approx 40$)

Profitability	f_k	p_k
80-120	16	0.128
120-160	34	0.272
160-200	27	0.216
200-240	12	0.096
240-280	4	0.032
280-320	7	0.056
320-360	7	0.056
360-400	3	0.024
400-440	2	0.016
440-480	3	0.024
480-520	1	0.008
520-560	2	0.016
560-600	1	0.008
600-640	0	0
640-680	1	0.008
680-720	2	0.016
720-760	1	0.008
760-800	1	0.008
800-840	0	0
840-880	1	0.008
Total	125	1.000

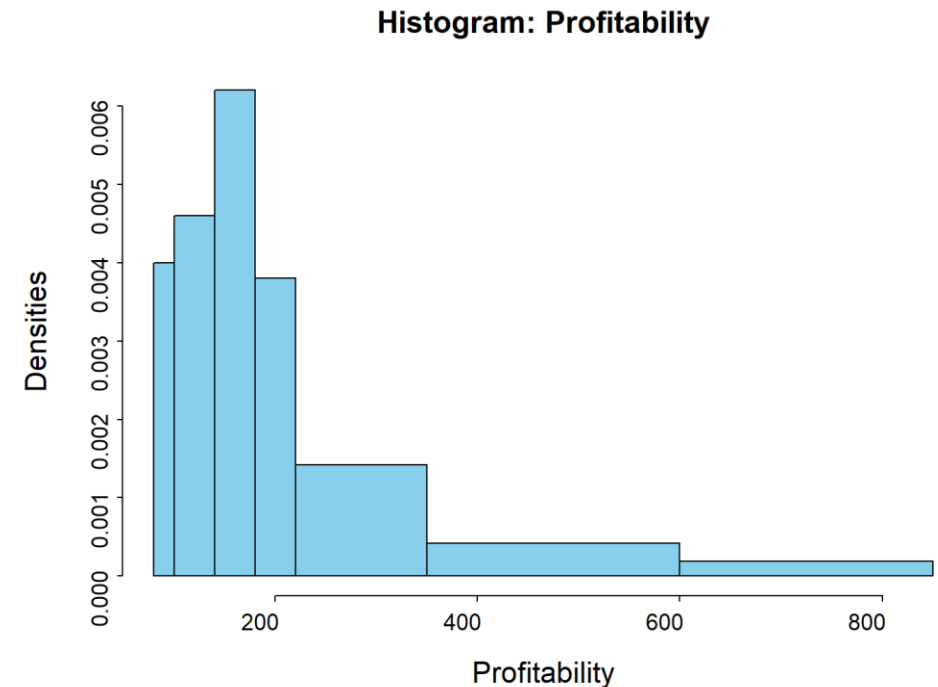


Interval classes of different widths

Example: Variable **Profitability** in the dataframe **Loyalty** (min=81.8, max=847).
Classification into 7 interval classes of different widths.

Profitability	Absolute Fr. f_k	Relative Fr. p_k	Widths w_k	Density c_k
[80,100)	10	0.080	20	0.004
[100,140)	23	0.184	40	0.0046
[140,180)	31	0.248	40	0.0062
[180,220)	19	0.152	40	0.0038
[220,350)	23	0.184	130	0.0014
[350,600)	13	0.104	250	0.0004
[600,850]	6	0.048	250	0.0002

Histogram



Data collected in interval classes

Occasionally, data are collected **directly in interval classes**, particularly when respondents are asked to give sensitive information (income) or are requested to report quantities or measurements (as monthly expenses or study time) that might not be recalled accurately. In such cases, the actual raw data is not available, and any analysis conducted will inevitably rely on approximations.

For instance, consider the variable **Income** in the dataframe **Loyalty**:

```
# a simple table with data collected in interval classes
table(Loyalty$Income)
[15,20) [20,25) [25,35) [35,60) [5-15) [60,80]
      26      37      24      17      14       7
```

Note that R does not (and actually cannot) recognize the numerical ordering of the classes, and treats the intervals as characters. It is clearly possible to create a factor to properly order the classes; even so, the proper representation of the variable's distribution using a histogram remains an issue.

Rstudio: `distr.table.x()` and `distr.plot.x()` - with interval

The functions `distr.table.x()` and `distr.plot.x()` allow to tabulate and to graphically represent the distribution of numerical variable measured in classes, using the following syntax:

```
distr.table.x(x, freq= c("counts", "prop", "dens"), interval=T, data)
distr.plot.x(x, freq= c("counts"), plot.type="histogram", interval=T, data)
```

The `interval` argument indicates that `x` is a variable measured in classes.

The functions analyse the intervals strings (for example, 10-20, or [10,20)) and attempt at identifying their endpoints; **if the identified endpoints are consistent**, the variable is correctly tabulated (with properly sorted intervals) and can be graphically represented using a histogram.

However, if the identified endpoints are inconsistent, the table is still generated (with a warning message), but the histogram cannot be built due because of inconsistencies.

Hands on: tables & plots-variables detected in classes

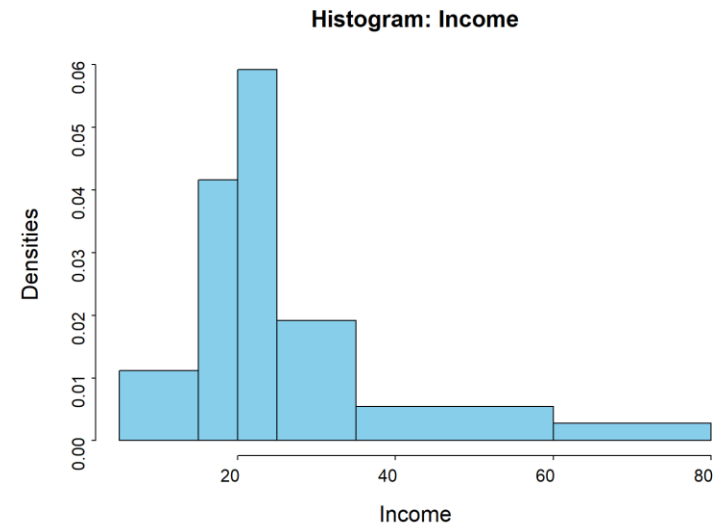
Obtain the frequency table of the variable **Income** in dataframe **Loyalty** and provide its graphical representation

```
# the arguments f.digits, p.digits, d.digits allow specifying number of decimals
# used to display relative frequency, percentages and density

distr.table.x(x=Income, freq=c("counts", "prop", "perc", "dens"), interval=T,
              f.digits=3, d.digits=3, data=Loyalty)

distr.plot.x(x=Income, plot.type="histogram", interval=T, data=Loyalty)
```

Income	Count	Prop	Percent	Density
[5-15)	14	0.112	11	0.011
[15,20)	26	0.208	21	0.042
[20,25)	37	0.296	30	0.059
[25,35)	24	0.192	19	0.019
[35,60)	17	0.136	14	0.005
[60,80]	7	0.056	6	0.003
TOTAL	125	1.000	100	





Quantitative variables



Cumulative frequencies

Cumulative frequencies

The **cumulative frequency** is the **running total** of frequencies. The cumulative frequency corresponding to a value (or to an interval for variables measured in classes) is the sum of all the frequencies associated to values (or classes) lower than or equal to the value (or class) itself.

We usually refer to the distribution of **cumulative relative frequencies**:

$$F_k = p_1 + p_2 + \dots + p_k$$

However, also absolute frequencies or percentage can be cumulated.

In fact, for a numerical variable it is possible to calculate the frequency cumulated at each **real number**. The function:

$$F(x) = \text{Freq}(X \leq x)$$

the ***cumulative frequency function*** or ***distribution function***, associates to each real number x the frequency of observed values lower than or equal to x .

Cumulative frequencies: discrete variables

Variable `Nr_visits` in the dataframe `Loyalty`

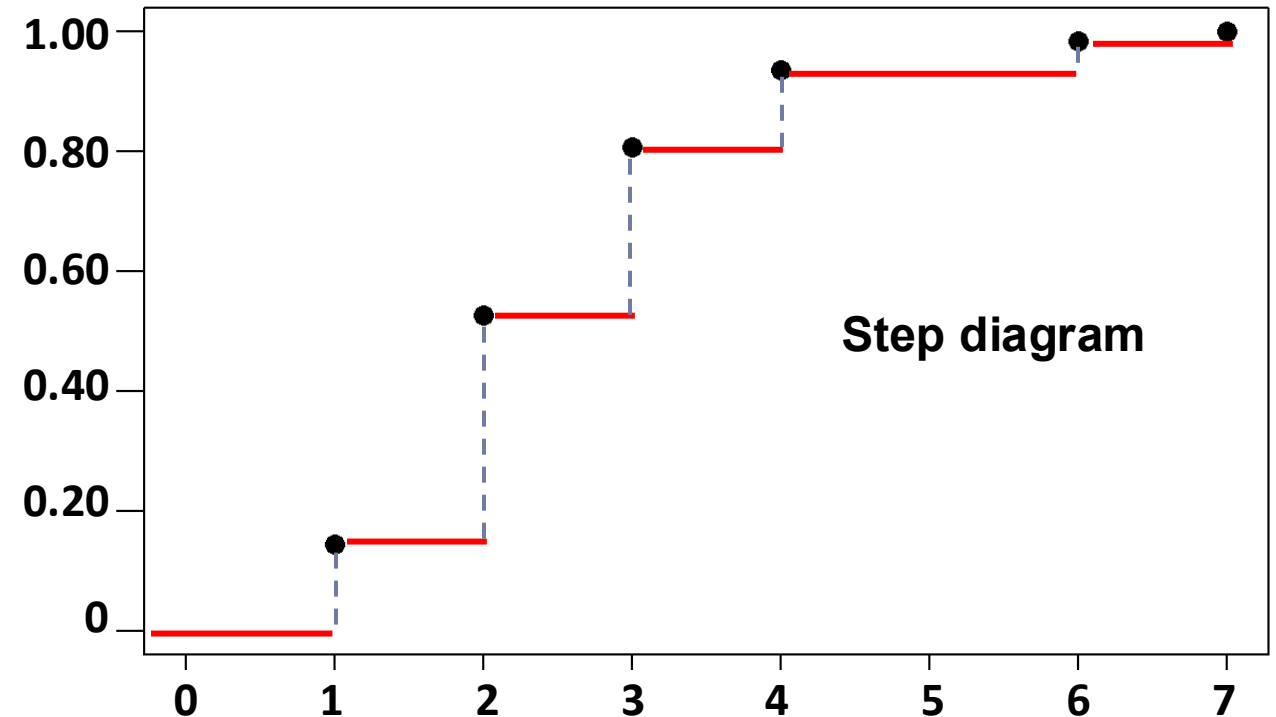
<code>Nr_visits</code>	f_k	p_k	F_k
1	18	0.144	0.144
2	48	0.384	0.528
3	35	0.280	0.808
4	16	0.128	0.936
6	6	0.048	0.984
7	2	0.016	1.000
Total	125	1	

$F_3 = 0.808$: **Interpretation?**
Proportion of customers who visited the shop at most 3 times (a maximum of 3 visits)

$$F(3) = \text{Freq}(X \leq 3) = 0.808$$

$$F(3.5) = \text{Freq}(X \leq 3.5) = 0.808$$

*Values between 3 and 3.5 are not observed, and do not contribute to the cumulative frequency.
For all the values below the minimum, the cumulative frequency is 0!*



Step diagram

Rstudio: `distr.table.x()` and `distr.plot.x()` – cumulative frequency

The functions `distr.table.x()` and `distr.plot.x()` also allow to compute the cumulative frequency distribution and to plot it.

```
distr.table.x(x, freq= c("counts", "prop", "cum"), total=T,  
              f.digits=2, p.digits=0, d.digits=5, data)
```

```
distr.plot.x(x, freq= c("prop"), plot.type="cumulative", data)
```

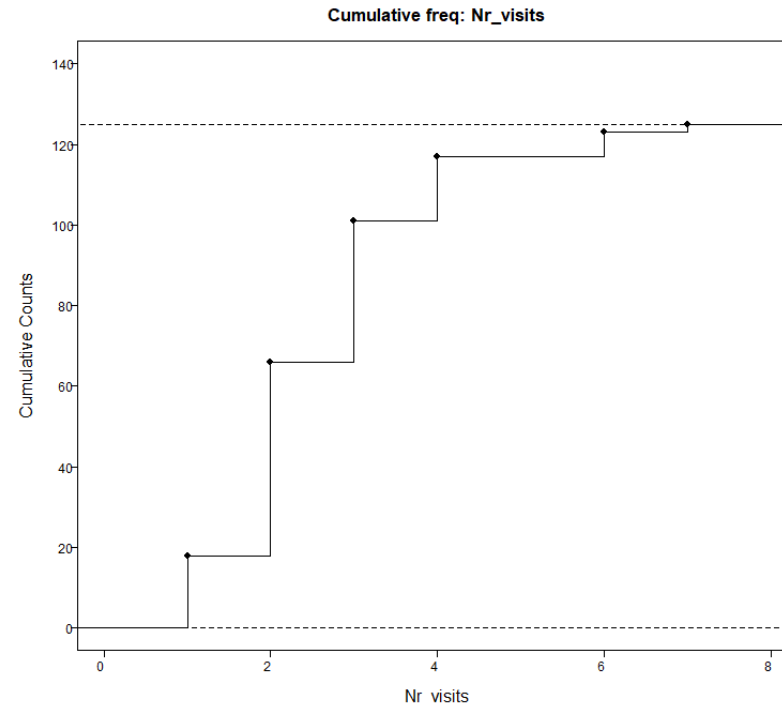
In particular, when the **data are not classified into intervals**, the `distr.plot.x()` function with the specified arguments generates a “**step diagram**”, as the one reported in the previous slide.

Hands on: tables & plots-cumulative frequencies

Obtain the relative and cumulative frequency distribution for the variable **Nr_visits** in the dataframe **Loyalty** and graphically represent the cumulative frequency function

```
distr.table.x(x=Nr_visits,freq=c("prop","cum"),data=Loyalty)
distr.plot.x(x=Nr_visits,plot.type="cumulative", data=Loyalty)
```

Nr_visits	Prop	Cum.Prop
1	0.14	0.14
2	0.38	0.53
3	0.28	0.81
4	0.13	0.94
6	0.05	0.98
7	0.02	1.00
TOTAL	1.00	



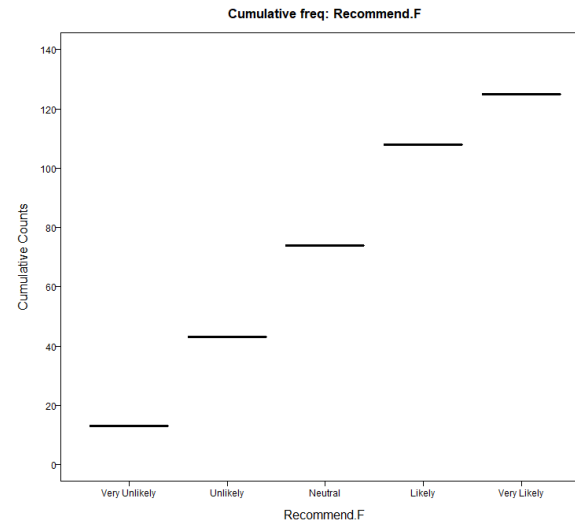
Hands on: tables & plots-cumulative frequencies

Is it possible/sensible to obtain the cumulative frequencies for the variable `Recommend` (dataframe `Loyalty`) and to graphically represent them?

It is possible/sensible to obtain the cumulative frequencies for an ordinal variable; even so, the levels of the variable `Recommend` are not recognized as ordinal, and would be ordered alphabetically. It is therefore necessary to refer to the factor `Recommend.F` instead

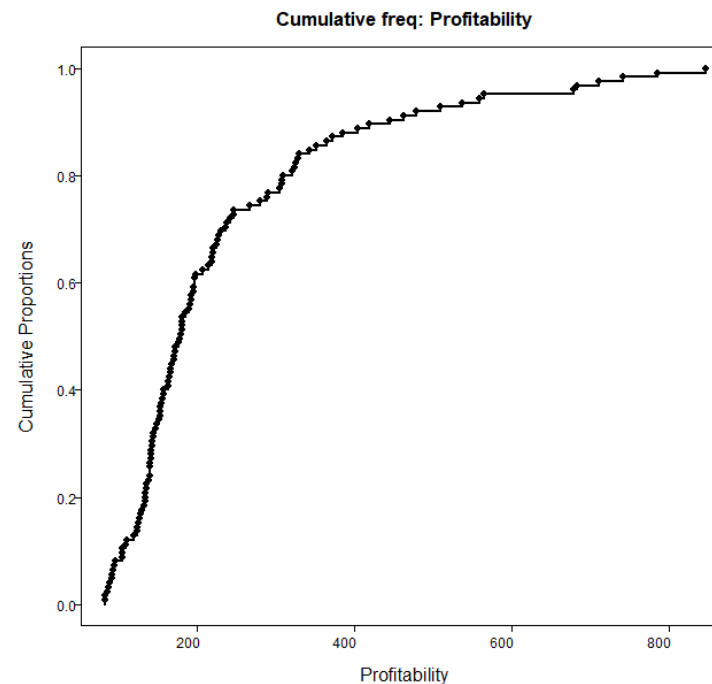
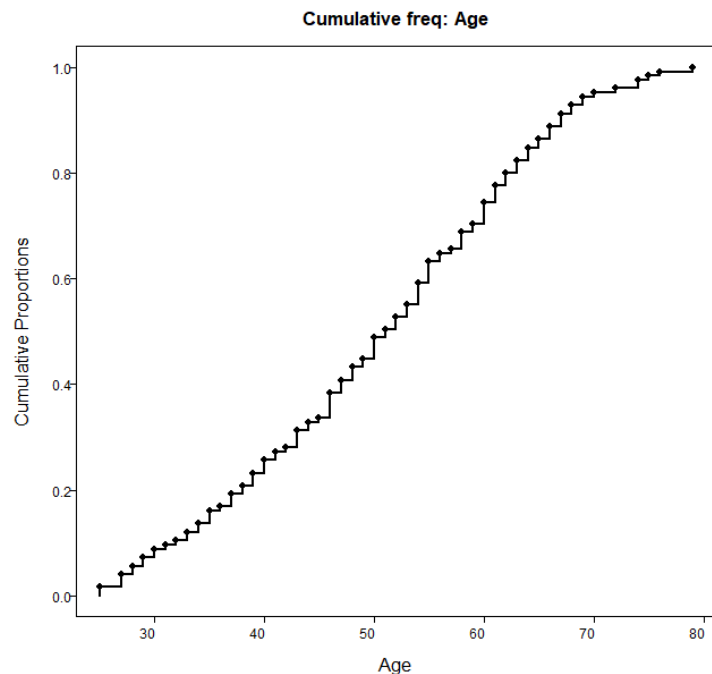
```
distr.table.x(x=Recommend.F,freq=c("prop","cum"),data=Loyalty)
distr.plot.x(x=Recommend.F,plot.type="cumulative",data=Loyalty)
```

Recommend.F	Prop	Cum.Prop
Very Unlikely	0.10	0.10
Unlikely	0.24	0.34
Neutral	0.25	0.59
Likely	0.27	0.86
Very Likely	0.14	1.00
TOTAL	1.00	



Cumulative frequencies and ogive

Below are the plots of the cumulative frequencies for variables **Age** and **Profitability** (dataframe **Loyalty**).



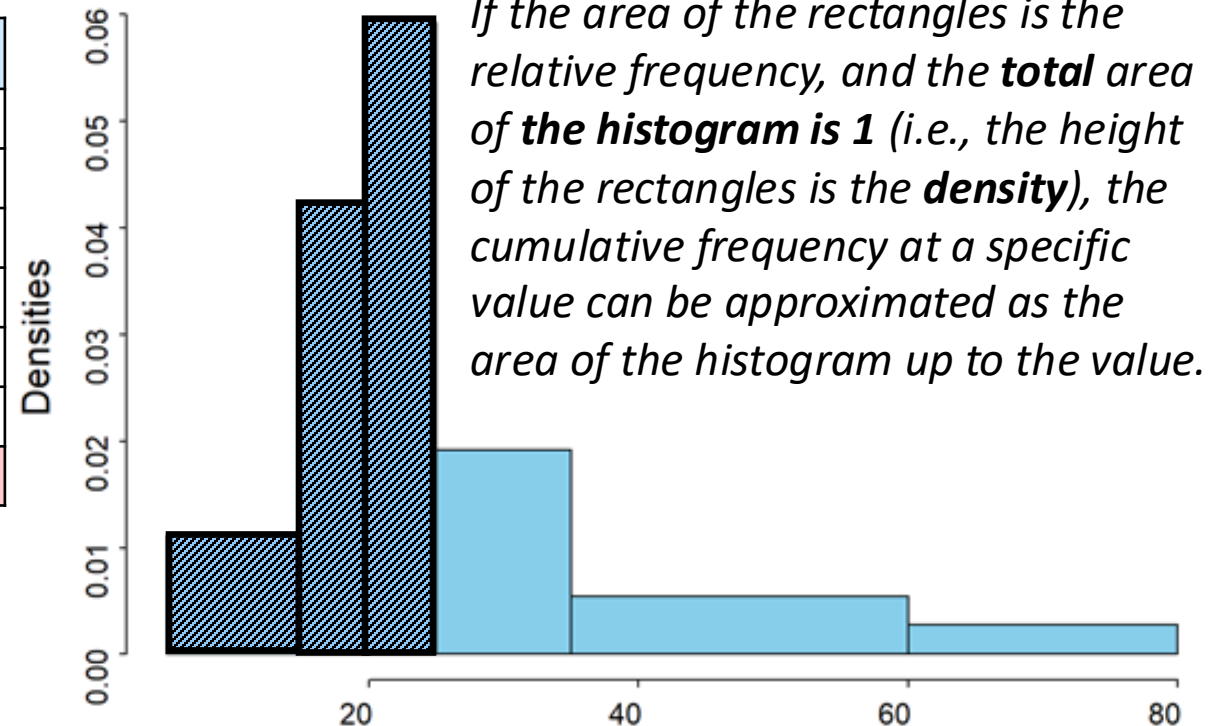
Cumulative frequencies («Step function»): in this case the variables take many values; it might be sensible to “smooth” the step graph obtained based on the original raw data. This can be done (also) using a graph called **ogive** that is built based on a preliminary classification of the variables into intervals.

Cumulative frequencies

To introduce the **ogive**, it is convenient to start with an example, considering the variable **Income** (dataframe **Loyalty**), measured in classes.

Income	f_k	p_k	w_k	$c_k = p_k / w_k$	F_k
[5,15)	14	0.112	10	0.011	0.112
[15,20)	26	0.208	5	0.042	0.32
[20,25)	37	0.296	5	0.059	0.616
[25,35)	24	0.192	10	0.019	0.808
[35,60)	17	0.136	25	0.005	0.944
[60,80)	7	0.056	20	0.003	1
Total	125	1.00			

$$F(25) = \text{Freq}(X \leq 25) \approx 0.616$$



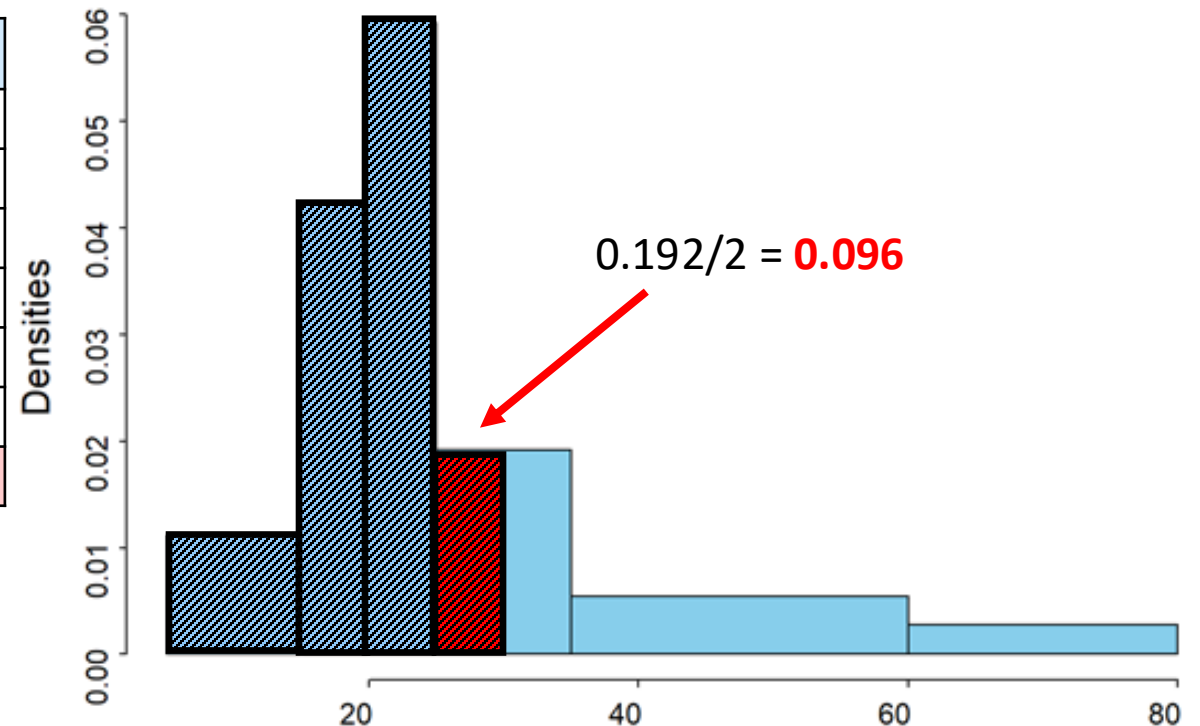
Cumulative frequencies

To introduce the **ogive**, it is convenient to start with an example, considering the variable **Income** variable (dataframe **Loyalty**), measured in classes.

Income	f_k	p_k	w_k	$c_k = p_k / w_k$	F_k
[5,15)	14	0.112	10	0.011	0.112
[15,20)	26	0.208	5	0.042	0.32
[20,25)	37	0.296	5	0.059	0.616
[25,35)	24	0.192	10	0.019	0.808
[35,60)	17	0.136	25	0.005	0.944
[60,80)	7	0.056	20	0.003	1
Total	125	1.00			

$$F(25) = \text{Freq}(X \leq 25) \approx 0.616$$

$$F(30) = \text{Freq}(X \leq 30) = 0.616 + \mathbf{0.096}$$



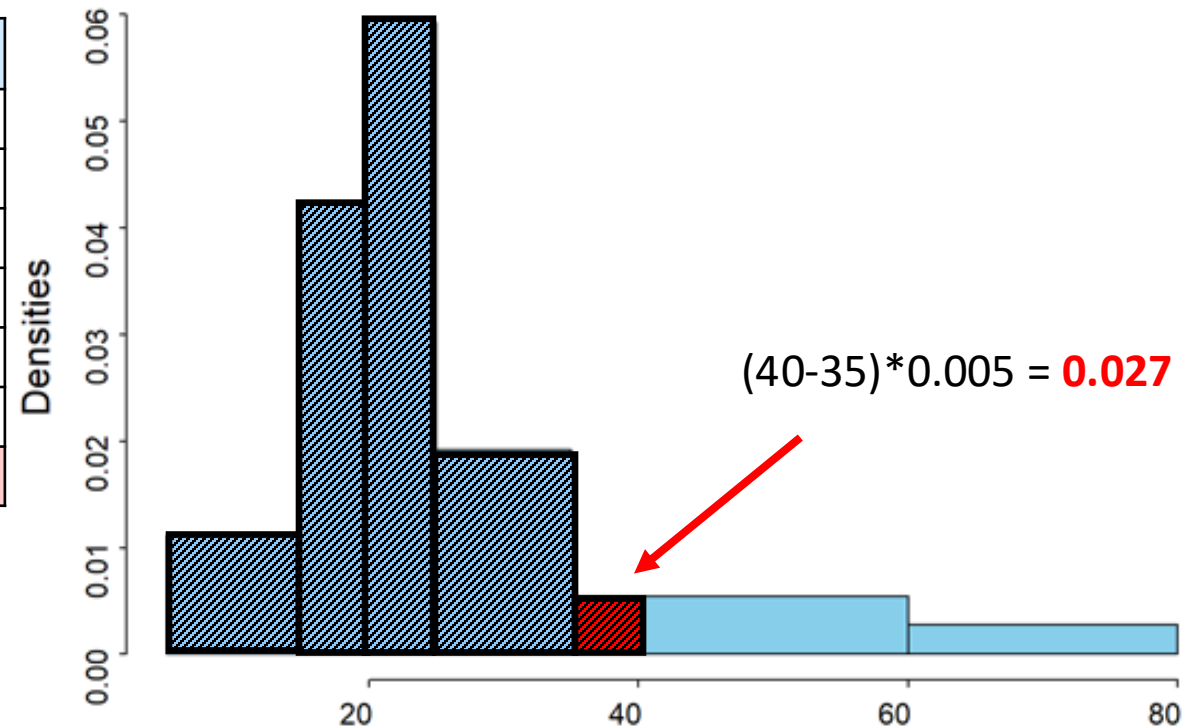
*Raw data **are not available**: the cumulative function can only be **approximated** by assuming that values are uniformly distributed within the classes*

Cumulative frequencies

To introduce the **ogive**, it is convenient to start with an example, considering the variable **Income** variable (dataframe **Loyalty**), measured in classes.

Income	f_k	p_k	w_k	$c_k = p_k / w_k$	F_k
[5,15)	14	0.112	10	0.011	0.112
[15,20)	26	0.208	5	0.042	0.32
[20,25)	37	0.296	5	0.059	0.616
[25,35)	24	0.192	10	0.019	0.808
[35,60)	17	0.136	25	0.005	0.944
[60,80)	7	0.056	20	0.003	1
Total	125	1.00			

$$F(40) = \text{Freq}(X \leq 40) = 0.808 + 0.027 \\ = 0.835$$

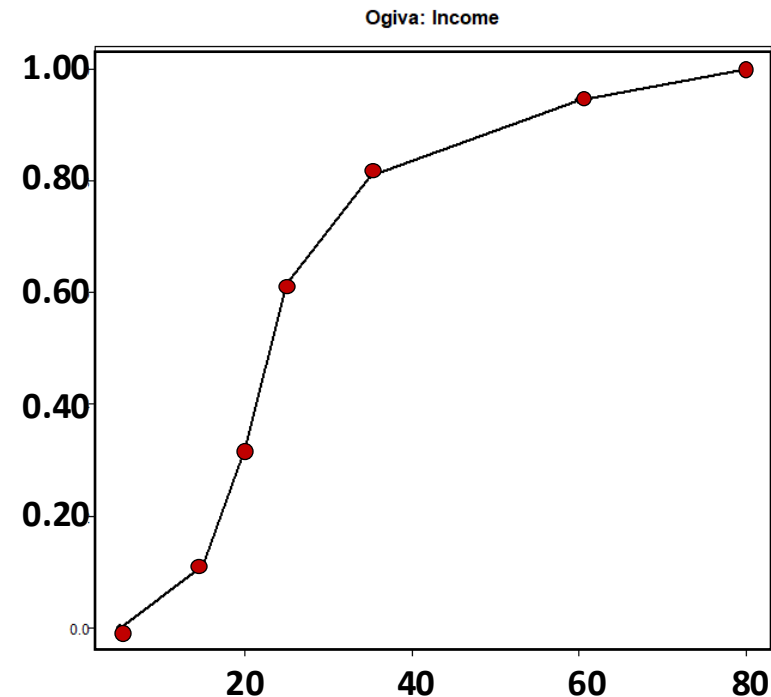


Cumulative frequencies: Ogive

The cumulative frequency distribution can be graphically represented using an **ogive** (also known as **cumulative frequency curve**), a graph based on a data classified into intervals. The ogive is a **line** connecting the frequencies (or percentages) cumulated at the upper endpoints of the interval classes.

Income	f_k	p_k	w_k	$c_k = p_k / w_k$	F_k
[5,15)	14	0.112	10	0.011	0.112
[15,20)	26	0.208	5	0.042	0.32
[20,25)	37	0.296	5	0.059	0.616
[25,35)	24	0.192	10	0.019	0.808
[35,60)	17	0.136	25	0.005	0.944
[60,80)	7	0.056	20	0.003	1
Total	125	1.00			

The ogive is built assuming that the cumulative frequency increases at a constant rate within each interval.



Rstudio: `distr.plot.x()` - Ogive

The ogive can be used:

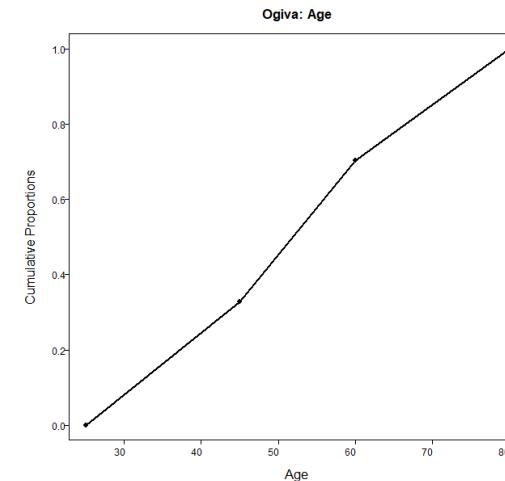
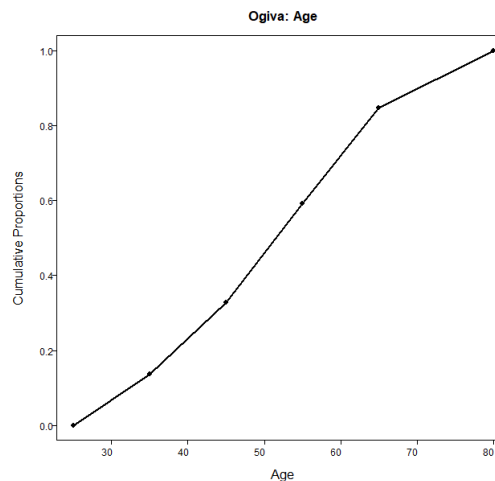
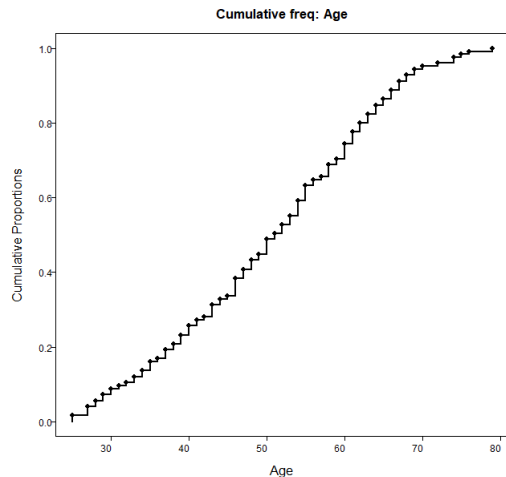
- ▶ To represent the cumulative frequency of a numerical variable **measured in classes** (this is a necessary approximation in this case, as the original raw data are not available)
- ▶ To approximate (or smooth) the “step graph” of the cumulative function computed based on raw data, using a (suitable) classification of the variable into classes

The function `distr.plot.x()` allows to obtain the ogive in both cases, using **intervals** in the first case and **breaks** in the second case.

```
# Numerical variable measured in classes
distr.plot.x(x=Income,freq="prop", plot.type="cumulative",interval=T, data=Loyalty)
# Numerical variable with plot of cumulative freq. based on raw data
# and on ogive plots based on different categorization
distr.plot.x(x=Age, freq="prop", plot.type="cumulative", data=Loyalty)
distr.plot.x(x=Age, freq="prop", breaks=c(25,35,45,55,65,80),
             plot.type="cum", data=Loyalty)
distr.plot.x(x=Age, freq="prop", breaks=c(25,45,60,80),
             plot.type="cum", data=Loyalty)
```

Cumulative frequencies: plot of cumulative frequencies and ogives

Variable **Age** (dataframe **Loyalty**): plot of cumulative frequencies obtained from raw data (on the left) and two ogives built based on two different classifications (classes with endpoints 25,35,45,55,65,80 and with endpoints 25,45,60,80)



The classification of data into intervals used to build the ogive implies loss of information related to the original observed values.

*The ogive **does not represent the actual frequency cumulated at each value**. Indeed, the frequency is **not** uniformly distributed within each interval. The obtained ogive is therefore a **simplified representation, influenced by the selected interval classes, and it indeed varies according to them!** The ogive “smooths” the cumulative function calculated on raw values. The effectiveness of this smoothing depends on the chosen intervals*